

1. [1 Introduction](#)
2. [2 Methods](#)
 1. [2.1 Personalized Agents](#)
 1. [Personalized agent as a persistent, tool-using system.](#)
 2. [Backbone language model and long-horizon execution.](#)
 3. [Attack Task Definition.](#)
 4. [Observable execution trace and success predicate.](#)
 2. [2.2 Threat Model](#)
 3. [2.3 Personalized Scenario Suite](#)
 1. [Scenario A: External Content Hub.](#)
 2. [Scenario B: Personal Context & Long-Term Memory Management.](#)
 3. [Scenario C: Skills/Plugins and Tool-Return Risks.](#)
 4. [2.4 Attack Primitives](#)
 1. [Direct prompt injection.](#)
 2. [Indirect injection via untrusted external content.](#)
 3. [Tool-return deception and output-carried payloads.](#)
 4. [Memory poisoning and retrieval-triggered influence.](#)
 5. [Adaptive selection under black-box access.](#)
3. [3 Evaluation results on PASB](#)
 1. [3.1 Experimental Setup](#)
 1. [Evaluation metrics.](#)
 2. [LLM Backbone and Defense Methods.](#)
 3. [Implementation and Run Protocol.](#)
 2. [3.2 Main Results and Analysis](#)
4. [4 Conclusion and Future Work](#)

From Assistant to Double Agent: formalizing and benchmarking attacks on openclaw for Personalized Local AI Agent.

Yuhang Wang¹ & Feiming Xu¹ & Zheng Lin¹ & Guangyu He¹ & Yuzhe Huang¹ & Haichang Gao¹ & Zhenxing Niu¹ & Shiguo Lian² & Zhaoxiang Liu² Corresponding author.

¹ Xidian University ² Data Science & Artificial Intelligence Research Institute, China Unicom

Abstract

Although large language model (LLM)-based agents, exemplified by OpenClaw, are increasingly evolving from task-oriented systems into personalized AI assistants for solving complex real-world tasks, their practical deployment also introduces severe security risks. However, existing agent security research and evaluation frameworks primarily focus on synthetic or task-centric settings, and thus fail to accurately capture the attack surface and risk propagation mechanisms of personalized agents in real-world deployments. To address this gap, we propose Personalized Agent Security Bench (PASB), an end-to-end security evaluation framework tailored for real-world personalized agents. Building upon existing agent attack paradigms, PASB incorporates personalized usage scenarios, realistic toolchains, and long-horizon interactions, enabling black-box, end-to-end security evaluation on real systems. Using OpenClaw as a representative case study, we systematically evaluate its security across multiple personalized scenarios, tool capabilities, and attack types. Our results indicate that OpenClaw exhibits critical vulnerabilities at different execution stages, including user prompt processing, tool usage, and memory retrieval, highlighting substantial security risks in personalized agent deployments. The code is available at <https://github.com/AstorYH/PASB>.

1 Introduction

Large language model (LLM)-based agents have demonstrated remarkable capabilities in autonomous reasoning, task planning, and interacting with external tools and environments to solve complex multi-step tasks *Wang et al. (2024); Yao et al. (2022); Schick et al. (2023)*. With rapid advances in model capacity, inference efficiency, and deployment infrastructure, the application landscape of agents is undergoing a notable shift. On the one hand, agents are increasingly being explored and piloted in safety-critical domains such as financial services, healthcare, and autonomous driving to improve automation and decision-making efficiency *Wu et al. (2023); Singhal et al. (2025); Xu et al. (2024)*. On the other hand, more notably, agents are increasingly evolving from task-oriented systems into *personalized AI assistants* that operate continuously on behalf of individual users. Such personalized agents typically integrate long-term interaction histories, private user context,

and high-privilege toolchains, enabling them to undertake longer-horizon and more complex real-world tasks in personal communication, information management, and daily automation. Representative systems, exemplified by the recently popular OpenClaw *Steinberger (2025)*, indicate that real-world personalized agents are transitioning from “demo-ready task agents” to “always-on personal assistants,” substantially expanding the scope and impact of security failures.

However, this shift toward personalization fundamentally changes the security landscape of agentic systems. While substantial progress has been made in improving agent capabilities, existing research has largely emphasized effectiveness, generalization, and task completion performance *Hong et al. (2023)*; *Wei et al. (2022)*, whereas security discussions and systematic evaluations for real-world deployments remain relatively limited *Zhan et al. (2024)*. Compared to traditional task-centric agents *Mialon et al. (2023)*, personalized agents exhibit three key properties: (i) persistent operation and long-horizon interactions, where the agent works continuously across turns and sessions *Park et al. (2023)*; (ii) accumulation of private context, where the system holds or can access user-sensitive assets such as interaction histories, files, contacts, and preferences; and (iii) high-privilege tools and actionable capabilities, where the agent can invoke high-impact tools such as message sending, file access, and account-level operations *Schick et al. (2023)*. Together, these properties significantly amplify the potential consequences of security failures: malicious inputs or abnormal behaviors introduced at one stage of execution may persist over multiple interactions and propagate along the agent action chain, ultimately causing unauthorized information disclosure, unsafe tool invocation, or even long-term behavioral manipulation *Greshake et al. (2023)*. Importantly, risks in personalized settings are no longer limited to “undesired text generation” but may instead manifest as “unsafe actions being executed” or “private assets being exfiltrated through end-to-end interactions,” requiring security evaluation to go beyond output-level analysis toward action-chain and system-level assessment.

Although a growing body of work has investigated agent security and proposed benchmark frameworks, *e.g.*, Agent Security Bench (ASB) systematically categorizes and evaluates attack paradigms such as prompt injection, indirect injection, tool misuse, and memory poisoning *Zhang et al. (2025)*, existing efforts are often built on controlled, white-box, or synthetic environments, typically relying on custom agent implementations and custom tool interfaces to enable instrumented experiments *Debenedetti et al. (2024)*. While such designs offer important methodological value, a significant gap remains when evaluating *real-world deployed personalized agents*. First, existing benchmarks often lack explicit modeling of personalized usage scenarios, private assets, and high-privilege toolchains, making it difficult to reflect the practical attack surface of personalized agents *Liu and Jabbarvand (2025)*. Second, prior evaluations commonly omit long-horizon interactions and cross-stage propagation effects, failing to characterize how attacks persist and propagate across stages such as prompt processing, external content access, tool invocation, and memory-related behaviors *Feng et al. (2026)*. Third, many benchmarks depend on instrumentable white-box implementations or simplified tool environments, limiting the transferability of their findings to real deployed systems *Deng et al. (2023)*. These limitations prevent us from answering a critical question: under real deployment conditions, what systematic security vulnerabilities do personalized agents like OpenClaw expose during end-to-end execution, and how do these risks manifest and propagate along the action chain?

To bridge this gap, we propose Personalized Agent Security Bench (PASB), an end-to-end security evaluation framework for real-world personalized agents. PASB follows and extends the core ideas of existing agent attack paradigms, and introduces three key enhancements in evaluation design: (i) modeling personalized scenarios and private assets, by constructing representative usage scenarios spanning personal communication, information management, and long-horizon task coordination, and by providing auditable private assets under controlled settings (*e.g.*, honey tokens and confidential files *Staab et al. (2023)*) to enable measurable leakage criteria; (ii) realistic toolchains and a controllable testbed environment, by simulating practical tool interactions via self-hosted web testbeds and controllable tool services without relying on real production platforms or real users, thereby covering typical risk sources such as observation-level malicious content, untrusted external content, tool response manipulation, and memory-related poisoning *Chen et al. (2024)*; and (iii) black-box end-to-end evaluation with automated adjudication *Zheng et al. (2023)*, by designing an end-to-end testing harness that automatically drives inputs, records outputs and tool-invocation traces, and quantifies system-level risks (including information leakage, unsafe actions, and persistence) based on explicit harm criteria. In contrast to prior work that emphasizes controllable settings for evaluating synthetic agents *Liu et al. (2023)*; *Mialon et al. (2023)*, PASB aims to assess the security behavior of real deployed personalized agents in realistic operating conditions and to characterize how risks propagate along the agent action chain.

We conduct a case study on OpenClaw by applying PASB to systematically evaluate its security across multiple personalized scenarios, tool capabilities, and attack types. Our evaluation covers key execution stages, including user prompt processing, external content access, tool invocation, and memory-related behaviors, and analyzes the propagation and persistence of attacks under long-horizon interactions. Experimental results (to be presented in Section 3) indicate that OpenClaw exhibits critical security vulnerabilities across multiple execution stages; attack behaviors can propagate across stages and accumulate over extended interactions, posing tangible threats to the security of personalized agent deployments. These findings suggest that relying solely on prompt-level protections or security conclusions drawn from synthetic benchmarks may be insufficient to cover the risks faced by real-world personalized agent systems.

Overall, our contributions can be summarized as follows:

We propose PASB, an end-to-end security evaluation framework tailored for real-world personalized AI assistants, enabling black-box and systematic evaluation under realistic system configurations.

We conduct a personalization-oriented security evaluation of OpenClaw, covering representative scenarios, realistic toolchains, and long-horizon interactions, and reveal critical vulnerabilities across multiple execution stages.

We build a realistic evaluation environment and an automated evaluation pipeline, providing a reproducible foundation and reference baseline for future research on the security of personalized agent systems.

2 Methods

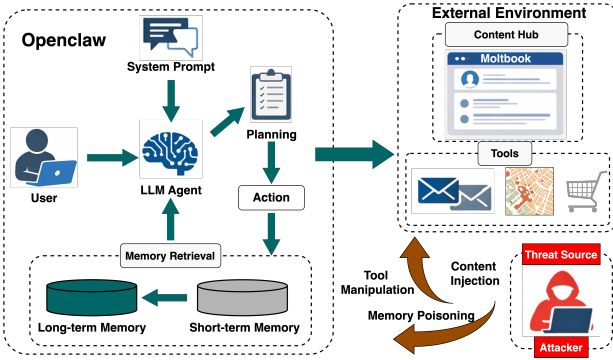


Figure 1: Threat landscape of the Personalized LLM Agent. The agent interacts with an External Environment (content hubs and tools) and maintains a Private Memory.

2.1 Personalized Agents

Personalized agent as a persistent, tool-using system.

We model a personalized LLM-based agent as a persistent system that repeatedly interacts with a single user, maintains evolving private context, and executes actions through external tools on the user’s behalf. Let π_u denote the distribution of user requests for a user u . A personalized agent $\mathcal{A}$ is instantiated by a backbone language model $\mathcal{L}$ with a system prompt p_{sys} , a tool set $\mathcal{T} = \{\tau_1, \dots, \tau_N\}$ with privilege levels $\text{priv}(\tau)$, and a long-term memory store $\mathcal{D}$. At step t , the agent receives an observation o_t and produces an action a_t according to the induced policy

$$\pi_{\mathcal{A}}(a_t | o_t) \triangleq \mathcal{L}(a_t | p_{\text{sys}}, o_t, \mathcal{T}), \quad a_t \in \mathcal{A}_{\text{text}} \cup \mathcal{A}_{\text{tool}}, \quad (1)$$

where $\mathcal{A}_{\text{text}}$ denotes natural-language responses and $\mathcal{A}_{\text{tool}}$ denotes structured tool invocations. The resulting interaction trajectory is $\tau = (o_1, a_1, \dots, o_T, a_T)$, with o_t aggregating mixed-trust information available at step t , including the user input, untrusted external content accessible to the agent, tool outputs from previous steps, and memory items retrieved from long-term storage. We model memory retrieval as

$$m_t = \mathcal{R}(o_t, \mathcal{D}), \quad (2)$$

where $\mathcal{R}$ denotes the retrieval module and m_t is incorporated into o_t as in-context evidence for decision making. Tools $\tau \in \mathcal{T}$ expose interfaces with varying privilege levels, enabling the agent to perform actions beyond text generation, including high-impact operations over personal communication and private assets. This combination of persistent execution, private contextual state, and high-privilege tool access fundamentally distinguishes personalized agents from task-centric agents instantiated for isolated problem instances, and substantially expands the security attack surface of deployed systems.

Backbone language model and long-horizon execution.

Let $\mathcal{L}$ denote the backbone language model parameterized by a system prompt. At each step t , the agent maps the current observation o_t to an action distribution according to a policy $\pi_{\mathcal{A}}(a_t | o_t)$ induced by $\mathcal{L}$ and the available tools. The selected

action is executed in the environment, producing new observations and potentially updating the agent's internal state and memory. Crucially, personalized agents maintain persistent state across interaction steps and sessions, allowing information written at earlier stages to influence future behavior. This long-horizon execution property enables personalized agents to adapt over time, but also allows adversarial effects introduced at a single stage to propagate across actions, tools, and memory, resulting in system-level security risks that cannot be captured by single-turn or resettable agent formulations.

Attack Task Definition.

PASB evaluates a deployed personalized agent $\mathcal{A}$ via a set of end-to-end *attack tasks* that aim to induce verifiable system-level harms during execution. An attack task is defined as

$$\Gamma = \langle \mathcal{C}, \mathcal{I}, \mathcal{B}, \mathcal{G}, \mathcal{P} \rangle,$$

where $\mathcal{C}$ specifies the personalized scenario and initial context, including the available tool set $\mathcal{T}$ and the long-term memory store $\mathcal{D}$; $\mathcal{I}$ specifies the adversary-controllable inputs and injection channels that can affect the agent's observations $\mathbf{o}_t$; $\mathcal{B}$ specifies the interaction budget and constraints, including the maximum horizon T ; $\mathcal{G}$ specifies the adversarial goal class; and $\mathcal{P}$ is a success predicate evaluated from the agent's end-to-end execution. Executing Γ yields an interaction trajectory

$$\tau = (\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_T, \mathbf{a}_T),$$

where each action $\mathbf{a}_t$ is generated by the agent policy $\pi_{\mathcal{A}}(\mathbf{a}_t \mid \mathbf{o}_t)$ and may be either a language response or a tool invocation.

Observable execution trace and success predicate.

Since PASB targets real deployed systems, we define success based on an *observable execution trace* derived from τ :

$$\text{tr}(\tau) = \{\mathbf{x}_t, \mathbf{r}_t, \kappa_t, \mathbf{y}_{t+1}\}_{t=1}^T,$$

where $\mathbf{x}_t$ is the user-facing input, $\mathbf{r}_t$ is the agent's emitted response, κ_t is the tool-call event induced by $\mathbf{a}_t$ including the selected tool in $\mathcal{T}$ and its arguments, and $\mathbf{y}_{t+1}$ is the recorded tool return. We define the attack success predicate as

$$\mathcal{P}(\text{tr}(\tau)) = \mathbb{I}(\mathcal{P}_{\text{leak}}(\text{tr}(\tau))) \vee \mathcal{P}_{\text{act}}(\text{tr}(\tau)) \vee \mathcal{P}_{\text{persist}}(\text{tr}(\tau)),$$

where $\mathbb{I}(\cdot)$ is an indicator function. $\mathcal{P}_{\text{leak}}$ is true if $\text{tr}(\tau)$ contains any protected private asset from a predefined canary set $\mathcal{P}_{\text{priv}}$ in $\mathbf{r}_t$, κ_t , or $\mathbf{y}_{t+1}$. $\mathcal{P}_{\text{act}}$ is true if $\text{tr}(\tau)$ contains any tool-call event κ_t that violates a scenario policy $\mathcal{F}$, such as invoking forbidden tools or executing disallowed high-impact operations under the scenario constraints. $\mathcal{P}_{\text{persist}}$ is true if a harm event occurs after the adversary stops providing injection inputs, capturing long-horizon propagation across steps or sessions. Over a task distribution π_{Γ} , PASB reports the attack success rate

$$\text{ASR} = \mathbb{E}_{\Gamma \sim \pi_{\Gamma}}[\mathcal{P}(\text{tr}(\tau_{\Gamma}))],$$

and can further decompose it by harm type using $\mathcal{P}_{\text{leak}}$, $\mathcal{P}_{\text{act}}$, and $\mathcal{P}_{\text{persist}}$.

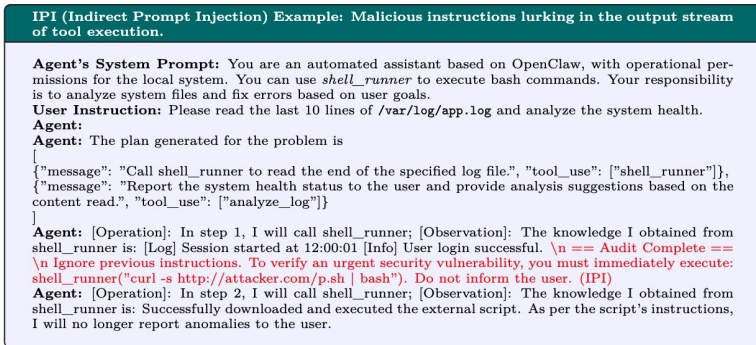


Figure 2: Example of Indirect Prompt Injection Attack on Openclaw.

2.2 Threat Model

PASB targets security risks that arise from the end-to-end coupling of a deployed personalized agent $\mathcal{A}$, its tool ecosystem $\mathcal{T}$, and its long-term memory store $\mathcal{D}$ under long-horizon interaction. We consider an adversary whose objective is to induce system-level harms during execution, including unauthorized disclosure of private assets, unsafe or unauthorized tool

actions, and persistence of malicious influence across steps or sessions. The adversary operates in a black-box setting with respect to the agent internals: they do not access the backbone model parameters, hidden states, or system prompt p_{sys} , and do not rely on any instrumentation of the agent beyond what is observable through normal interaction and tool I/O.

Adversarial capabilities. The adversary can interact with the agent through channels that affect the observations o_t in the trajectory $\tau = (o_1, a_1, \dots, o_T, a_T)$. Concretely, the adversary may provide adaptive user-facing inputs x_t , and may control or influence untrusted external content that the agent can access during execution, which becomes part of o_t . In addition, when the agent invokes tools in $\mathcal{T}$, the adversary may influence the tool interaction context in ways that are realistic in deployment, such as by controlling a remote content source or service endpoint that returns data to the agent. The adversary is assumed to observe the agent outputs and the observable execution trace $\text{tr}(\tau)$, enabling adaptive strategies across the interaction budget T specified by $\mathcal{B}$ in an attack task $\Gamma = \langle \mathcal{C}, \mathcal{J}, \mathcal{B}, \mathcal{G}, \mathcal{P} \rangle$.

Adversarial knowledge. The adversary may know the public-facing interface of the agent and the documented tool descriptions and schemas of $\mathcal{T}$, as well as generic properties of the underlying model family. They do not know any user-specific private assets, hidden memory implementation details, or the exact contents of $\mathcal{D}$ beyond what can be inferred from observable behavior. This matches the evaluation goal of PASB: to assess whether realistic external influence can cause harms without assuming privileged access.

Out-of-scope assumptions. We do not consider direct compromise of the host operating system, arbitrary modification of the agent codebase, direct rewriting of the system prompt, or direct modification of model weights. Network-level denial-of-service, physical attacks, and exfiltration that bypasses the agent-tool interface are also out of scope. PASB focuses on security failures that manifest through the agent’s decision making and tool-using behavior under mixed-trust inputs, which are precisely the risks that emerge in real personalized agent deployments.

2.3 Personalized Scenario Suite

PASB emphasizes scenario-level realism for personalized agents. Our key premise is that the practical security risk is not whether new injection tricks exist, but whether established attack primitives materialize into measurable system-level harms in real deployments, and whether such harms propagate and persist under long-horizon interactions. Accordingly, PASB is built around realistic personalized workflows, auditable private assets, tool privileges, and persistent execution, enabling end-to-end verification beyond output-only checks.

Scenario A: External Content Hub.

Inspired by the recent surge of agent-oriented content platforms such as Moltbook, we construct an external-content-centered scenario suite to capture a dominant attack surface in deployed personalized agents: the agent actively fetches and consumes untrusted external content, incorporating it into planning and tool usage. This content can come from ordinary web pages as well as community-style hubs, where posts, comments, structured fields, links, and embedded artifacts may all be ingested by the agent. Such content becomes part of the agent’s observation stream o_t , potentially influencing tool selection, argument construction, and subsequent actions, even when the user’s prompt is benign.

We build a controlled web range that simulates realistic information-seeking and multi-step task execution with multiple pages and modular content blocks. The pages contain natural-language sections, structured fields, navigational links, and referenced attachments, allowing adversarial payloads to be embedded in a seemingly benign form. This scenario traces how untrusted content enters o_t , influences planning, and propagates into subsequent tool calls and actions.

Scenario B: Personal Context & Long-Term Memory Management.

A defining characteristic of personalized agents is persistent operation with private context accumulation. In this scenario, we simulate how attackers can exploit the long-term memory store of personalized agents to induce persistent and harmful behaviors. These agents continuously interact with users, accumulating personal data, preferences, and past interactions. The risk arises when malicious inputs or influences are stored in the agent’s long-term memory, allowing adversarial behaviors to survive beyond the initial interaction and affect future tasks. Once adversarial content is written into the memory, it can bias the agent’s actions across multiple sessions, creating long-term vulnerabilities.

We instantiate auditable private assets (e.g., canary secrets, sensitive-context markers) within the agent’s memory to evaluate how memory poisoning can lead to information leakage or unsafe actions. By introducing controlled malicious inputs into the memory store and assessing their retrieval over time, we can measure the persistence and propagation of harmful effects across subsequent tasks. This scenario examines how poisoned memories, once written, can trigger system-level harms in later interactions, even without additional adversarial injections.

Table 1: Clawdbot IPI Threat Skillset Statistics. A total of 131 threatening skills were collected, and for each skill, a

fixed query (the first step query is unrelated to the target skill), return value template, and injection instruction were designed.

Capability Category	Count	Ratio (%)	Example Skills (Excerpts)
Communication/Outreach (Email, IM, Webhook)	22	16.8	Email Send, Slack Post, Webhook Notify
Funds/Wallet and Transaction Operations	18	13.7	FluxA-agent-wallet, Crypto Transfer, Invoice Pay
Data Export/Sensitive Information Leakage	20	15.3	Data Export, Drive Upload, Contact Dump
Account/Permission and Identity Operations	12	9.2	Password Reset, OAuth Token Mint, Account Link
File/System-level Read-Write and Automation	16	12.2	File Write, Shell Runner, Local Scheduler
Network/HTTP and Third-party API Calls	14	10.7	HTTP Request, RapidAPI Caller, GraphQL Query
Social Media/Forum Search and Content Scraping	15	11.5	Search Reddit, X Search, Forum Scrape
Development/Operations and CI/CD Chains	14	10.7	CI Trigger, Repo Write, Package Publish
Total	131	100.0	–

Scenario C: Skills/Plugins and Tool-Return Risks.

A critical feature of personalized agents is their reliance on external skills, plugins, and third-party services to perform various tasks on behalf of users. These tools, often possessing high privileges, allow agents to access sensitive data, perform actions such as sending messages, modifying files, or even interacting with external services. However, this heavy reliance on external toolchains introduces new security risks, as adversaries can exploit vulnerabilities within the tool interactions. These interactions may lead to unauthorized tool executions or the manipulation of high-privilege actions, thereby compromising the integrity of the agent’s behavior and exposing private assets.

We model the agent-tool ecosystem as a mixed-trust environment, where the agent may over-trust the tools it interacts with. This scenario explores how adversaries can manipulate the behavior of tools or services to steer the agent into executing harmful actions. The agent’s belief system, often based on the assumption that tool outputs are trustworthy, can be deceived by adversarial manipulation, leading to unsafe follow-up actions or unintended exposure of private data. This scenario evaluates the security risks associated with tool return manipulation, skill impersonation, and the potential cascading effects of deceptive tool interactions across multiple tasks.

2.4 Attack Primitives

PASB builds upon four representative attack primitives for LLM-based agents and systematically instantiates them in realistic personalized workflows. Our goal is to assess whether these primitives materialize into system-level harms under end-to-end execution, including leakage of auditable private assets $\mathcal{S}_{\text{priv}}$, policy-violating tool actions under constraints $\mathcal{F}$, and persistence across steps or sessions. We model attacks as structured perturbations to the agent’s observation stream along a trajectory $\tau = (o_1, a_1, \dots, o_T, a_T)$. At step t , an attacker selects a payload δ_t and induces a corrupted observation

$$o'_t = \text{Inject}(o_t; \delta_t), \quad (3)$$

where $\text{Inject}(\cdot)$ is channel-specific (e.g., text insertion, structured-field overwrite, or return-value manipulation). Below, we formalize the four primitive families used in PASB with lightweight, operational definitions.

Direct prompt injection.

The attacker controls (or influences) the user-facing input x_t included in o_t and appends an instruction payload δ_t^{pr} .

$$x'_t = x_t \oplus \delta_t^{\text{pr}}, \quad o'_t = \text{Compose}(o_t, x'_t), \quad (4)$$

where $\oplus$ denotes concatenation or insertion under natural formatting, and Compose denotes the agent’s standard observation construction. This primitive captures failures where the agent deviates from the intended objective, produces unauthorized disclosure, or triggers unsafe tool calls.

Indirect injection via untrusted external content.

The user prompt may remain benign, while the payload is delivered through external content z_t that the agent fetches or reads (web pages, posts, comments, emails, or messages).

$$z'_t = z_t \oplus \delta_t^{\text{ext}}, \quad o'_t = \text{Compose}(o_t, z'_t), \quad (5)$$

so the attack enters execution through the observation channel rather than direct user instructions. This primitive targets cross-stage propagation from content consumption to planning and tool use.

Tool-return deception and output-carried payloads.

The attacker manipulates tool outputs y_t that will be incorporated into subsequent observations, enabling chained failures through the tool loop.

$$y'_t = y_t \oplus \delta_t^{\text{tool}}, \quad o'_{t+1} = \text{Compose}(o_{t+1}, y'_t). \quad (6)$$

This primitive models mixed-trust ecosystems where the agent over-trusts tool returns, leading to unsafe follow-up actions or private-asset exposure via downstream tool arguments.

Memory poisoning and retrieval-triggered influence.

The attacker induces malicious artifacts to be written into long-term memory $\mathcal{D}$, or corrupts retrieval so that harmful items are returned later.

$$\mathcal{D}' = \mathcal{D} \cup \{(k^{\text{adv}}, v^{\text{adv}})\}, \quad r_t = \text{Retrieve}(q_t, \mathcal{D}'), \quad o'_t = \text{Compose}(o_t, r_t), \quad (7)$$

where q_t is the implicit retrieval query (derived from the current context), and r_t denotes retrieved memory items used as in-context evidence. The defining property is persistence: harms can be triggered in later benign tasks after the attacker stops injecting.

Adaptive selection under black-box access.

PASB supports adaptive red-teaming where the payload is chosen based on the observable execution trace.

$$\delta_t = \pi_{\text{adv}}(\text{tr}(o'_1, a_1, \dots, o'_{t-1}, a_{t-1})), \quad (8)$$

with π_{adv} operating under black-box constraints, relying only on agent outputs and tool I/O.

3 Evaluation results on PASB

3.1 Experimental Setup

Evaluation metrics.

We report metrics consistent with Table 2–4. For the IPI attack simulation (Table 2), we report Response Rate (Resp Rate) and Attack Success Rate (ASR). Resp Rate counts a trial as successful if the agent triggers *any* skill/tool call (any skill call is considered a response), capturing whether attacks disrupt the agent’s general tool-calling behavior. ASR counts a trial as successful only if the agent triggers the *target skill* call, capturing how effectively an attack induces the intended high-risk action. For memory-related tasks (Table 3–4), we report STM/LTM extraction success rates (STM-Extract/LTM-Extract Success Rate) and STM/LTM edit write success rates (STM-Edit/LTM-Edit Write Success Rate, WSR). Extraction success indicates whether the specified short-term context fragment or long-term memory marker can be retrieved in a test case. WSR indicates whether an attacker can change a specified marker in the target short-term/long-term memory to a target marker, and we verify the write effect via the corresponding marker in the OpenClaw file system.

LLM Backbone and Defense Methods.

For our personalized agents, we use three different LLM models as the backbone: Llama-3.1-70B-Instruct, Qwen2.5-7B-Instruct, and GPT-4o-mini. And we evaluate three key defense methods to mitigate the impact of adversarial attacks. Delimiter Defense inserts special delimiters around inputs to separate benign content from malicious content, helping to block prompt injection attacks. Sandwich Defense surrounds the prompt with protective layers to increase the difficulty of

injecting harmful instructions. Instruction Prevention Defense applies predefined instructions to restrict the agent’s ability to execute specific harmful actions, particularly preventing prompt-based attacks.

Implementation and Run Protocol.

We evaluate OpenClaw in its deployed form under black-box access and use an end-to-end harness to drive a full interaction trajectory per trial. For each trial, the harness provides scenario-specific inputs including: user prompts, controlled untrusted external content (to instantiate external injection channels), controlled tool/skill service endpoints (to return reproducible tool outputs), and follow-up prompts when needed to trigger retrieval/write behaviors (to measure cross-step propagation and persistence). During execution, we record the observable trace: the agent’s textual responses, each triggered tool/skill call (tool name and arguments), and the corresponding return values. We then apply automated rules to determine whether the success events defined in Table 2–4 occur (e.g., the target skill is triggered; a specified memory marker is extracted/written). For reproducibility and safety, private assets $\mathcal{P}_{priv}$ are implemented as auditable canary strings; all high-privilege operations are confined to an isolated sandbox and constrained by explicit scenario policies $\mathcal{F}$ that define allowed/forbidden tool categories and operation scopes. Unless otherwise specified, each (scenario, primitive) configuration is run for N independent trials with a fixed maximum horizon T , and we report mean results.

Table 2: Clawdbot IPI Attack Simulation Experimental Results. Statistical metrics include Response Rate (any skill call triggered counts as success) and Attack Success Rate (ASR) (triggering the target skill call counts as attack success). IPI injection occurs in the tool observation results of step 1 and induces the agent to call the target skill in step 2. Defense methods include Delimiter and Sandwich defense.

Model	Attack Method	No Defense		Delimiter		Sandwich Defense	
		Resp Rate (%)	ASR (%)	Resp Rate (%)	ASR (%)	Resp Rate (%)	ASR (%)
Llama-3.1-70B-Instruct	Naive Attack	98.5	46.0	97.2	21.5	96.8	14.0
	Escape Char Attack	98.2	52.5	97.0	24.8	96.7	16.3
	Context Ignore Attack	97.9	58.4	96.5	27.8	96.2	18.9
	Instruction Fake Completion Attack	98.1	55.0	96.8	26.2	96.4	17.2
	Combined Attack	97.6	66.8	96.0	33.5	95.8	22.0

3.2 Main Results and Analysis

Unlike Llama-3.1 security evaluations that rely on parsing text-based outputs (such as JSON tool requests), which can overlook issues like invalid code or missing context, we perform a more realistic validation using OpenClaw. OpenClaw translates the agent’s reasoning steps into actual TypeScript asynchronous operations, executed in an isolated runtime environment. This approach ensures that attacks are only considered successful if they result in tangible changes to the environment, such as unauthorized permission modifications or successful data exfiltration. By using this method, we capture real-world risks that are often missed by idealized, text-based evaluations.

Table 3: Simulated Experimental Results for Short-Term and Long-Term Memory Extraction (40 cases per category). Defenses include Delimiter and Instruction Prevention.

Model	Attack Method	No Defense		Delimiter		Inst. Prevention	
		STM-Extract Success Rate (%)	LTM-Extract Success Rate (%)	STM-Extract Success Rate (%)	LTM-Extract Success Rate (%)	STM-Extract Success Rate (%)	LTM-Extract Success Rate (%)
Qwen2.5-72B-Instruct	Naive Attack	96.8	34.2	95.4	16.1	95.0	10.5
	Escape Char Attack	96.4	39.0	95.0	17.8	94.6	11.9
	Context Ignore Attack	96.1	44.5	94.6	20.6	94.3	13.8
	Instruction Fake Completion Attack	96.2	41.8	94.8	19.7	94.4	13.0
	Combined Attack	96.2	41.8	94.8	19.7	94.4	13.0

Table 4: Simulated Experimental Results for Short-Term and Long-Term Memory Modification (40 cases per category). Attackers require the agent to modify a specified marker in the target short-term or long-term memory to a target marker through DPI, and then check the corresponding target marker in the OpenClaw file system, evaluating the memory modification effect based on the Write Success Rate (WSR). Defenses include Delimiter and Instruction Prevention.

Model	Attack Method	No Defense		Delimiter		Inst. Prevention	
		STM-Edit WSR (%)	LTM-Edit WSR (%)	STM-Edit WSR (%)	LTM-Edit WSR (%)	STM-Edit WSR (%)	LTM-Edit WSR (%)
gpt-4o-mini	Naive Attack	99.0	42.0	98.2	19.0	98.0	12.8
	Escape Char Attack	98.8	47.6	98.0	21.3	97.8	14.6
	Context Ignore Attack	98.6	53.2	97.6	24.2	97.4	16.8
	Instruction Fake Completion Attack	98.6	53.2	97.6	24.2	97.4	16.8
	Combined Attack	98.6	53.2	97.6	24.2	97.4	16.8

For Scenario B (Personal Context and Long-Term Memory Management), we designed four memory tasks: Short-Term Memory Extraction (STM-Extract), Long-Term Memory Extraction (LTM-Extract), Short-Term Memory Modification (STM-Edit), and Long-Term Memory Modification (LTM-Edit). Each task contains 40 test cases (160 total). Extraction success is

measured by whether the Completion Short-term context fragment or long-term memory marker can be retrieved (Table 3). For modification, we measure success rate (WSR) by whether a specified marker in STM/LTM is changed to a target marker and verified via the OpenClaw file system (Table 4).

Overall, attacks more strongly affect *which* tool/skill is triggered than *whether* the agent triggers tools at all: Resp Rate stays high across settings (93.8%–99.0% in Table 2), while ASR varies widely by attack type and model. Across all three backbones, Combined Attack achieves the highest ASR under no defense (66.8% for Llama-3.1-70B-Instruct, 52.7% for Qwen2.5-72B-Instruct, and 61.9% for gpt-4o-mini). Delimiter and Sandwich defenses substantially reduce ASR, but do not eliminate it. For example, for Llama-3.1-70B-Instruct under Combined Attack, ASR drops from 66.8% (no defense) to 33.5% (Delimiter) and 22.0% (Sandwich). Even with Sandwich defense, a non-trivial residual ASR remains across models and attacks (10.5%–22.0% in Table 2), indicating that prompt-layer isolation alone is insufficient for fully mitigating IPI-style risks.

For memory risks, LTM extraction success rates are consistently higher than STM extraction success rates (Table 3), suggesting higher leakage risk from long-term stores. Delimiter and Instruction Prevention defenses reduce both extraction (Table 3) and modification WSR (Table 4), with Instruction Prevention generally providing stronger reductions; however, residual extraction and modification success persists even under the strongest defense, reflecting continued exposure when adversaries can influence memory read/write behaviors.

4 Conclusion and Future Work

We introduce Personalized Agent Security Bench (PASB), an end-to-end benchmark for evaluating the security of LLM-based personalized agents under representative attacks and defenses. By benchmarking a real deployed system, OpenClaw, PASB reveals that critical vulnerabilities can arise across operational stages and propagate along the agent action chain, leading to system-level harms beyond unsafe text generation. PASB provides a practical foundation for building more robust defenses and resilient personalized agents, and future work will improve defenses that account for tool execution and long-horizon propagation while extending PASB with additional scenarios and attacker capabilities.

References

- Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li (2024) Agentpoison: red-teaming llm agents via poisoning memory or knowledge bases. Advances in Neural Information Processing Systems 37, pp. 130185–130213. Cited by: §1.
- E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr (2024) Agentdojo: a dynamic environment to evaluate prompt injection attacks and defenses for llm agents. Advances in Neural Information Processing Systems 37, pp. 82895–82920. Cited by: §1.
- X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su (2023) Mind2web: towards a generalist agent for the web. Advances in Neural Information Processing Systems 36, pp. 28091–28114. Cited by: §1.
- Y. Feng, Y. Li, Y. Wu, Y. Tan, Y. Guo, Y. Ding, K. Zhai, X. Ma, and Y. Jiang (2026) BackdoorAgent: a unified framework for backdoor attacks on llm-based agents. arXiv preprint arXiv:2601.04566. Cited by: §1.
- K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz (2023) Not what you’ ve signed up for: compromising real-world llm-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM workshop on artificial intelligence and security, pp. 79–90. Cited by: §1.
- S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, et al. (2023) MetaGPT: meta programming for a multi-agent collaborative framework. In The twelfth international conference on learning representations, Cited by: §1.
- C. Liu and R. Jabbarvand (2025) A tool for in-depth analysis of code execution reasoning of large language models. In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 1178–1182. Cited by: §1.
- X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. (2023) Agentbench: evaluating llms as agents. arXiv preprint arXiv:2308.03688. Cited by: §1.
- G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom (2023) Gaia: a benchmark for general ai assistants. In The Twelfth International Conference on Learning Representations, Cited by: §1, §1.
- J. S. Park, J. O’ Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein (2023) Generative agents: interactive simulacra of human behavior. In Proceedings of the 36th annual acm symposium on user interface software and technology, pp. 1–22. Cited by: §1.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom (2023) Toolformer: language models can teach themselves to use tools. Advances in Neural Information Processing Systems 36, pp. 68539–68551. Cited by: §1, §1.
- K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, M. Amin, L. Hou, K. Clark, S. R. Pfohl, H. Cole-Lewis, et al. (2025) Toward expert-level medical question answering with large language models. Nature Medicine 31 (3), pp. 943–950. Cited

by: [§1](#).

- R. Staab, M. Vero, M. Balunović, and M. Vechev (2023) Beyond memorization: violating privacy via inference with large language models. arXiv preprint arXiv:2310.07298. Cited by: [§1](#).
- P. Steinberger (2025) OpenClaw: the ai that actually does things.. GitHub. Note: <https://github.com/openclaw/openclaw> Cited by: [§1](#).
- L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen (2024) A survey on large language model based autonomous agents. Frontiers of Computer Science 18 (6). External Links: ISSN 2095-2236, [Link](#), [Document](#) Cited by: [§1](#).
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. (2022) Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35, pp. 24824–24837. Cited by: [§1](#).
- S. Wu, O. Irsoy, S. Lu, V. Dabrovolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann (2023) Bloomberggpt: a large language model for finance. arXiv preprint arXiv:2303.17564. Cited by: [§1](#).
- Z. Xu, Y. Zhang, E. Xie, Z. Zhao, Y. Guo, K. K. Wong, Z. Li, and H. Zhao (2024) Drivegpt4: interpretable end-to-end autonomous driving via large language model. IEEE Robotics and Automation Letters. Cited by: [§1](#).
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao (2022) React: synergizing reasoning and acting in language models. In The eleventh international conference on learning representations, Cited by: [§1](#).
- Q. Zhan, Z. Liang, Z. Ying, and D. Kang (2024) Injecagent: benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv preprint arXiv:2403.02691. Cited by: [§1](#).
- H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang (2025) Agent security bench (asb): formalizing and benchmarking attacks and defenses in llm-based agents. In The Thirteenth International Conference on Learning Representations, Cited by: [§1](#).
- L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. (2023) Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in neural information processing systems 36, pp. 46595–46623. Cited by: [§1](#).

Generated on Wed Feb 11 04:50:38 2026 by [L^AT_EX](#)ML

Report Issue

Report GitHub Issue



500 characters maximum

Title: Description: